Implementation of a hybrid Brent-kung and Carry Select Adder for a multi-precision adder.

Emilio Segovia Salas

Ilias Karakonstantis

Summary:

The system is parameterized with an operand width of 512 and supports a maximum adder size of 32 bits. Even though it uses 32-bit adders, it processes 64 bits per cycle by calculating the result of the addition twice, once for each possible carry-in bit. The correct result is then selected through a multiplexer.

An iterative FSM is used to sequentially process the operands in chunks. To complete a full 512-bit addition, this implementation requires 8 clock cycles. The result is 513 bits wide, including the carry-out.

Technical Description:

First, within each 32-bit block, a Brent-Kung algorithm is used to compute at a logarithmic time.

The Brent – Kung algorithm is a scheme for binary addition that uses two operators called “Generate” and “Propagate”. For each bit i, these formulas are calculated in the following way:

$$G_i = A_i * B_i \; and \; P_i = A_i \bigoplus B_i$$

These formulas state if a carry is created or passed along. We then use the associative prefix operator:

$$(G_k , P_k) \circ (G_j , P_j) = (G_k + P_k G_j , P_k P_j)$$

This equation combines adjacent bit pairs into groups in terms of generate/propagate signals. Using this formulation, the carry at position i can be expressed as:

$$C_i = C_{i-1} + P_{i-1} G_{i-2} + \cdots + (P_{i-1} \ldots P_0) C_0$$

After the 32-bit Brent–Kung stage, a carry-select mechanism is used to eliminate carry propagation dependency. The lower 32-bit block is computed using the actual carry-in, while the upper 32-bit block is computed twice in parallel: once assuming a carry-in of 0 and once assuming a carry-in of 1. Once the lower block produces the correct carry-out, a multiplexer selects the appropriate upper result. This approach removes the need to wait for carry resolution.

After the multiplexing stage, the selected upper sum is concatenated with the lower sum to form a 64-bit result for the current cycle. The corresponding carry-out is stored in final_cout and used as the carry-in for the next cycle. Simultaneously, the 64-bit result is inserted into the output register through a shift-and-accumulate operation. This process is repeated iteratively, allowing the result to build progressively. The input operands are also shifted to expose the next 64-bit segments.

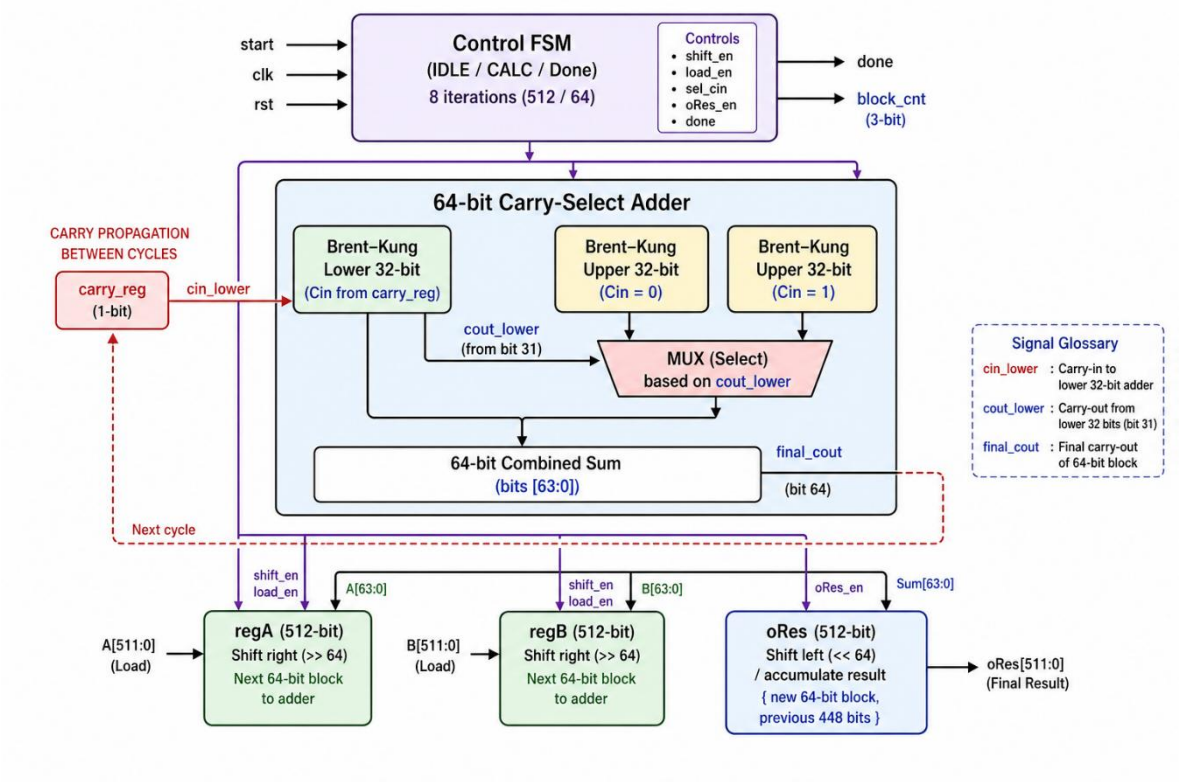


Figure 1 High level diagram of the algorithm.

Performance:

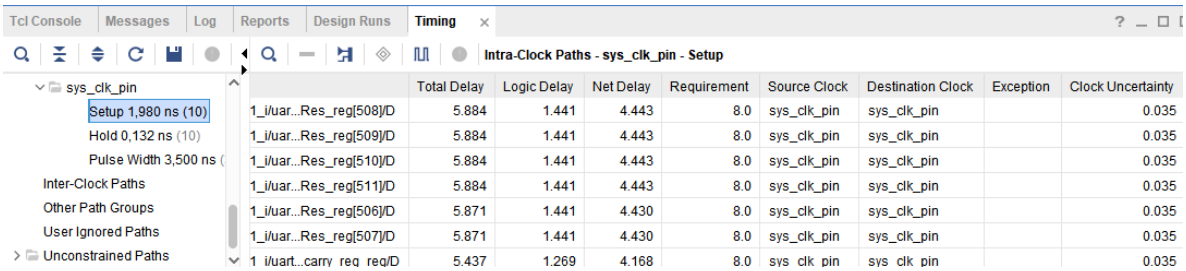


Figure 2 Worst case Total Delay of 5.884ns (1.441 + 4.443) for our hybrid model.

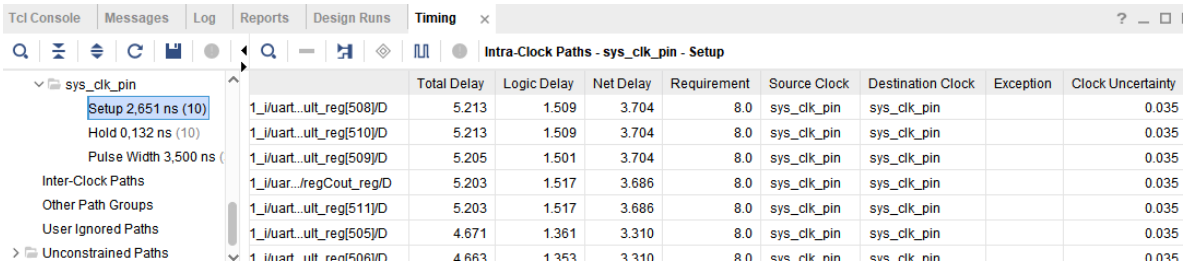


Figure 3 Worst case Total Delay of 5.213 (1.509 + 3.704) for the ripple carry adder.

In Figures 2 and 3, we observe that the worst case total delay of our hybrid design is higher than that of the ripple carry adder. This is expected, as the complexity of our system is greater than that of a simple ripple carry design. At 125 MHz, approximately 8 ns are available per clock cycle. Our design requires about 5.88 ns, which is within the allowable timing budget. Although it takes longer per cycle, the hardware remains active for a greater portion of time.

Additionally, we measured the number of clock cycles required by both systems. The ripple carry adder requires 34 cycles, while our design requires only 8 cycles. The maximum theoretical latency is therefore:

$$Maximum\ Theoretical\ Latency\ for\ RCA = 34 * 5.213 = 177.24ns$$

$$Maximum\ Theoretical\ Latency\ for\ the\ hybrid\ Model = 8 * 5.884 = 47.07ns$$

This shows that our system is approximately 3.77 times faster than a standard ripple carry adder in terms of total latency.

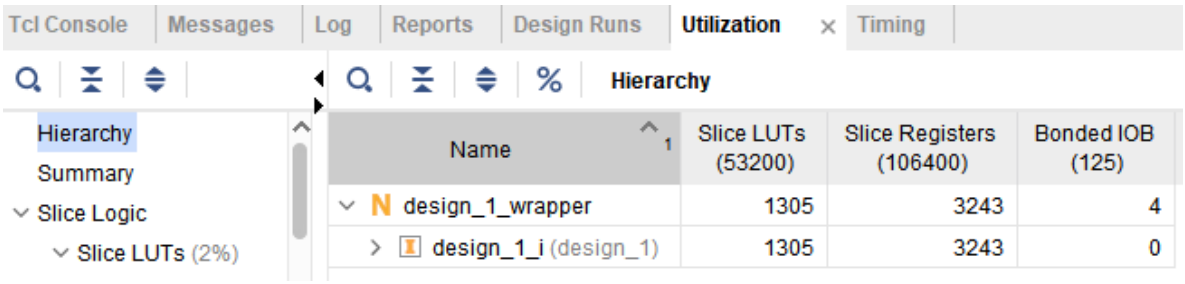


Figure 4 Area Cost of 1305 LUTs and 3243 FFs for our implementation.

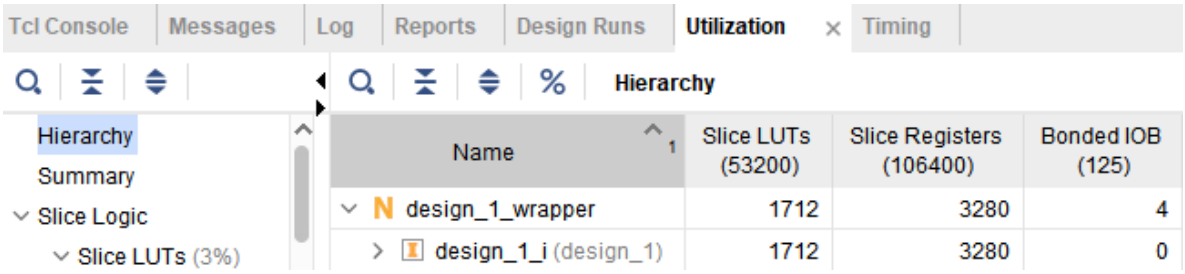


Figure 5 Area Cost of 1712 LUTs and 3280 FFs for a standard ripple carry adder.

From the utilization reports, the hybrid design uses 1305 LUTs compared to 1712 LUTs for the ripple carry adder, indicating lower combinational hardware usage. This is somewhat surprising, as the Brent–Kung structure is typically heavy in combinational logic, which is also reflected in the higher net delay.

Regarding slice registers, both implementations are similar (3243 vs. 3280), which is expected since they require comparable storage resources.

To summarize the performance segment in this report, we have observed an important tradeoff between the RCA and our model. Our model shows an overall reduction in area and overall latency; however, it requires longer critical path per cycle. While the ripple carries adder benefits from optimized carry chained software, achieving faster clock speeds but requiring more resources.